Guía de Laboratorio: Fundamentos de Sistemas Embebidos

Plataforma de trabajo: STM32F411E-DISCO
Entorno de desarrollo: STM32CubeIDE (Librerías HAL Cube)

Esta guía propone una serie de desafíos de complejidad ascendente. El objetivo principal es guiar al estudiante en la transición desde un paradigma de programación secuencial y bloqueante hacia la concurrencia cooperativa y preemptiva, herramientas fundamentales en el diseño de firmware robusto.

Disposición de Hardware (STM32F411 Discovery)

- **Botón de Usuario (User Button):** Pin `PA0` (Activo en Alto).
- **LED Verde:** Pin `PD12`.
- **LED Naranja:** Pin `PD13`.
- **LED Rojo:** Pin `PD14`.
- **LED Azul:** Pin `PD15`.

Desafíos Prácticos

Desafío 1: Memoria de estado y Antirrebote (Bloqueante)

Debounce por Software | HAL_Delay

Objetivo: Detectar flancos (transiciones) en lugar de niveles lógicos y lidiar con el rebote mecánico (*bouncing*) de los contactos.

Descripción: Al presionar el botón de usuario (`PA0`) una sola vez, el LED Rojo (`PD14`) debe cambiar de estado (*Toggle*) y mantenerse así. Dado que el pulsador genera ruido eléctrico al hacer contacto, se debe implementar una rutina de antirrebote simple esperando unos milisegundos tras detectar el primer flanco.

Pista de implementación:

```
if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0) == GPIO_PIN_SET) {
    HAL_Delay(50); // Antirrebote bloqueante

    if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0) == GPIO_PIN_SET) {
        HAL_GPIO_TogglePin(GPIOD, GPIO_PIN_14);

        // Esperar a que se suelte el botón para evitar múltiples toggles
        while(HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0) == GPIO_PIN_SET);
```

Desafío 2: Liberando la CPU (No Bloqueante con SysTick)

Temporización Cooperativa | HAL_GetTick()

Objetivo: Romper la dependencia de `HAL_Delay()` para permitir que el *super-loop* se ejecute continuamente sin interrupciones o pausas artificiales.

Descripción: Hacer parpadear el LED Azul (`PD15`) a 2 Hz (250 ms ON, 250 ms OFF). Simultáneamente, el botón debe encender el LED Verde (`PD12`) **solo** mientras esté presionado.

El problema a resolver: Si se utiliza `HAL_Delay()` para controlar el parpadeo del LED Azul, la respuesta a la lectura del botón será lenta y errática. El sistema debe utilizar el contador de milisegundos del sistema para planificar el cambio de estado del LED sin detener el flujo del programa.

Pista de implementación:

```
uint32_t currentMillis = HAL_GetTick();

if (currentMillis - previousMillis >= 250) {
    previousMillis = currentMillis;
    HAL_GPIO_TogglePin(GPIOD, GPIO_PIN_15);
}

// La lectura del botón se evalúa libremente en cada ciclo
if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0) == GPIO_PIN_SET) {
    HAL_GPIO_WritePin(GPIOD, GPIO_PIN_12, GPIO_PIN_SET);
} else {
    HAL_GPIO_WritePin(GPIOD, GPIO_PIN_12, GPIO_PIN_RESET);
}
```

Desafío 3: Máquina de estados finitos (Integración)

State Machine | Super-loop

Objetivo: Construir una lógica secuencial basada en estados, integrando entradas asíncronas y temporización no bloqueante.

Descripción: El sistema tendrá 3 estados definidos que cambian de manera cíclica cada vez que se presiona el botón:

- **Estado 0:** Todos los LEDs apagados.
- **Estado 1:** Parpadeo lento (1 Hz) de todos los LEDs simultáneamente.
- **Estado 2:** Parpadeo rápido (5 Hz) de todos los LEDs simultáneamente.

Requisito estricto: No se permite el uso de `HAL_Delay()` en el *super-loop*, a excepción (opcionalmente) de unos pocos milisegundos requeridos para el antirrebote durante la transición de estado. El tiempo de respuesta a la presión del botón debe ser inmediato independientemente de la frecuencia de parpadeo actual.

Temporizadores de Hardware (Timers)

Desafío 4: Precisión por Hardware (Timers - Polling)

Prescaler & ARR | Update Event Flags

Objetivo: Desvincular la base de tiempos del procesador central (SysTick) y utilizar un periférico de temporización dedicado.

Descripción: Crear un efecto visual de secuencia ("Coche Fantástico": Verde → Naranja → Rojo → Azul → Rojo → Naranja...) donde el cambio de cada LED esté gobernado por el desbordamiento de un Timer de hardware (por ejemplo, `TIM2`) configurado a 100 ms.

El procesador no debe contar tiempo; solo debe verificar mediante *polling* si la bandera de desbordamiento se levantó, limpiarla y avanzar a la siguiente etapa de la secuencia.

Pista de implementación:

```
// Tras iniciar el periférico con HAL_TIM_Base_Start(&htim2);

if (__HAL_TIM_GET_FLAG(&htim2, TIM_FLAG_UPDATE) != RESET) {
    __HAL_TIM_CLEAR_FLAG(&htim2, TIM_FLAG_UPDATE);

    // Ejecutar avance de la secuencia de LEDs
    AvanzarSecuenciaLEDs();
}
```

Desafío 5: Temporización en Segundo Plano (Interrupciones)

Concurrencia Preemptiva | NVIC & ISR callbacks

Objetivo: Introducir la concurrencia preemptiva real, permitiendo que el hardware interrumpa el flujo principal para ejecutar una tarea crítica en tiempo.

Descripción: Configurar un Timer (`TIM3`) para generar una interrupción exacta cada 500 ms. Dentro de la rutina de atención a la interrupción (ISR), alternar el estado del LED Naranja. Para demostrar la independencia del proceso, colocar en el `while(1)` principal un retardo masivo (ej. `HAL_Delay(5000);`) o un bucle computacionalmente pesado. El LED Naranja debe mantener su parpadeo a 1 Hz con precisión inalterable.

Pista de implementación:

```
// En el main(), antes del while(1):
HAL_TIM_Base_Start_IT(&htim3);

// Callback a definir fuera del main() (ej. final del archivo):
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    if (htim->Instance == TIM3) {
        HAL_GPIO_TogglePin(GPIOD, GPIO_PIN_13);
```

Desafío 6: Antirrebote Profesional (Muestreo en ISR)

Asynchronous Polling | volatile

Objetivo: Utilizar las interrupciones de temporizador para resolver problemas físicos (ruido de contactos), delegando el esfuerzo de procesamiento fuera del *super-loop* principal.

Descripción: Configurar un Timer (`TIM4`) para interrumpir el sistema cada 10 ms. En el callback de este timer, leer el estado del pulsador (`PA0`). Si el pulsador es leído en estado ALTO durante 5 interrupciones consecutivas (50 ms de estabilidad garantizada), se considera un flanco válido y la ISR debe levantar una bandera (*flag*).

El *super-loop* principal solo debe verificar esta bandera; si es verdadera, altera el estado del LED Azul y limpia la bandera. Esto libera al código principal de lógicas de retardo.

Pista de implementación:

```
volatile uint8_t button_pressed_flag = 0; // Obligatorio el uso de volatile

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    if (htim->Instance == TIM4) {
        static uint8_t press_counter = 0;

        if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0) == GPIO_PIN_SET) {
            press_counter++;
            if (press_counter == 5) {
                button_pressed_flag = 1; // Notificar al super-loop
            }
        } else {
            press_counter = 0; // Reiniciar si se detecta ruido o se soltó
        }
    }
}

// En el super-loop (main):
// if (button_pressed_flag) {
//      button_pressed_flag = 0;
//      HAL_GPIO_TogglePin(GPIOD, GPIO_PIN_15);
// }
```

📝 Nota de Cátedra sobre la Memoria Volátil

En el Desafío 6, es crítico comprender el uso de la palabra reservada `volatile` en C. Cuando declaramos la variable `button_pressed_flag` , le estamos indicando al compilador que su valor puede cambiar en cualquier momento sin previo aviso (en este caso, modificado por una interrupción de hardware subyacente).

Si se omite este calificador, el optimizador del compilador podría asumir erróneamente que, dado que el *super-loop* no modifica la variable de forma evidente, su valor es siempre cero. Esto